

MALAVIYA NATIONAL INSTITUTE OF TECHNOLOGY JAIPUR

Department of Computer Science and Engineering

Course: Cryptography Laboratory (22CPP307) | Group: 10 (Even Group Number)

Students: Rishi (2024UCP1566) | Nandini Verma (2024UCP1667)

Lab Assignment 7: Padding Oracle Attack on AES-CBC (Keyless Plaintext Recovery)

Target: AES-CBC Ciphertext with PKCS#7 Padding | **Key Access:** None (Zero Key Access)

Oracle Signal: Boolean (Valid / Invalid PKCS#7 Padding) | **Verification:** 100% Exact Plaintext Recovery

1. Theoretical Foundation and Cryptographic Mechanics

Cipher Block Chaining (CBC) Decryption: In AES-CBC mode, each ciphertext block C_i is decrypted through the block cipher primitive and XORed with the preceding ciphertext block C_{i-1} (with $C_0 = IV$):

$$P_i = \text{AES_DEC}(C_i) \oplus C_{i-1}$$

PKCS#7 Padding: Messages are padded to a 16-byte boundary by appending N bytes each of numerical value N (where $N \in [1, 16]$). If the data length is a multiple of 16, a full block of $16 \times 0x10$ is appended.

The Padding Oracle Vulnerability: A padding oracle is an endpoint or cryptographic service that decrypts a submitted ciphertext and responds with a single boolean bit: whether the resulting plaintext terminates in syntactically valid PKCS#7 padding. Because the attacker controls C_{i-1} , bit-flipping in C_{i-1} exerts deterministic, single-byte control over P_i . By querying the oracle across candidate bytes, the attacker recovers the intermediate state $I_i = \text{AES_DEC}(C_i)$ and thereby derives the entire plaintext P_i without knowing the secret AES key.

2. Mathematical Formulation & Right-to-Left Recovery Algorithm

Let $I_{i[j]} = \text{AES_DEC}(C_i)[j]$ be the j -th intermediate byte ($0 \leq j \leq 15$). Plaintext byte is $P_{i[j]} = I_{i[j]} \oplus C_{i-1}[j]$.

The recovery proceeds backwards from byte 15 (last byte) to byte 0 (first byte):

Step 1 (Target Pad Value): For byte position j , target padding value is $\text{pad} = 16 - j$.

Step 2 (Fix Trailing Bytes): For all already recovered positions $k > j$, set $C'[k] = I_{i[k]} \oplus \text{pad}$.

Step 3 (Brute-Force Candidate): Test candidate values $c \in [0, 255]$ at $C'[j]$ and submit $[C', C_i]$ to the oracle.

Step 4 (State Derivation): Upon receiving True: $I_{i[j]} = c \oplus \text{pad}$, yielding plaintext $P_{i[j]} = I_{i[j]} \oplus C_{i-1}[j]$.

Disambiguation for pad = 1: To prevent false positives from accidental multi-byte padding patterns (e.g. $0x02\ 0x02$), byte $C'[14]$ is mutated and re-queried when $\text{pad} = 1$.

3. Experimental Evaluation Across Test Scenarios

Scenario / Message Label	Blocks	CT Bytes	Oracle Queries	Worst-Case Max	Efficiency Gain	Recovery Match
Group 10 Assigned Message	4	64 B	8,142	16,384	50.3%	100% Exact
Short Message (Single Block)	1	16 B	1,959	4,096	52.2%	100% Exact
Exactly Two Blocks (32 B)	2	32 B	4,136	8,192	49.5%	100% Exact
Long Multi-Block Message	9	144 B	18,963	36,864	48.6%	100% Exact

4. Recovered Plaintext & Query Budget Analysis

Recovered Plaintext (Group 10 Dataset):

"CryptoLabX Group 10 Secret Message for Padding Oracle Lab!"

Verification: Exact character-by-character match against original plaintext without using the secret key.

Complexity and Query Budget Analysis:

- **Worst-Case Bound:** For N blocks of size B = 16 bytes, the exhaustive query upper bound is $256 \times B \times N = 4096 \times N$ queries.
- **Average-Case Expectation:** Assuming uniformly distributed intermediate byte values, finding each candidate requires an expected 128 queries per byte, yielding an average budget of $128 \times 16 \times N = 2048 \times N$ queries.
- **Measured Performance:** The experimental query count (8,142 queries across 4 blocks) corresponds to an average of 127.2 queries per byte, aligning within 0.6% of theoretical statistical expectation and demonstrating an efficiency gain of ~50% over worst-case brute force.

5. Prevention Strategies & Secure Architecture Recommendations

Countermeasure Strategy	Mechanism & Security Justification	Effectiveness
1. Authenticated Encryption (AEAD) (AES-GCM / ChaCha20-Poly1305)	Combines confidentiality and authenticity into a single cryptographic primitive. Decryption verifies the GHASH/Poly1305 MAC tag <i>before</i> processing any ciphertext blocks. Tampered ciphertexts abort instantly, completely eliminating padding inspection side-channels.	100% Immune (Industry Standard)
2. Encrypt-then-MAC (HMAC-SHA256)	Computes HMAC over the ciphertext using a distinct key. The server computes and validates the MAC in constant time (via <code>hmac.compare_digest</code>) before invoking decryption. Unauthenticated payloads never reach the decryption pipeline.	Highly Effective
3. Constant-Time Error Handling	Eliminates distinguishable error messages ('Invalid Padding' vs 'Decryption Error'). All validation failures return an identical generic error code with uniform execution latency to defeat timing side-channel attacks.	Defense-in-Depth
4. Constant-Time Padding Check	Validates PKCS#7 padding by bitwise OR accumulation over the entire block without early returns or conditional branch jumps, mitigating microarchitectural and timing-based oracle leaks.	Algorithmic Hardening
5. Migration to TLS 1.3	TLS 1.3 completely deprecated CBC mode ciphersuites (RFC 8446), mandating AEAD-only ciphersuites and eliminating padding oracle vulnerabilities at the protocol layer.	Architectural Solution

6. Conclusion & Verification Summary

This laboratory demonstrates that confidentiality guarantees in block ciphers depend strictly on message integrity. CBC mode without authentication leaks full plaintext when paired with an observable padding error signal. Full mitigation requires modern Authenticated Encryption (AES-GCM) or strict Encrypt-then-MAC verification.